

Tutorial 8: Data Cleaning and Transformation

1. Introduction

Data cleaning and transformation are essential steps in the **data preprocessing stage** of data analysis and machine learning. Real-world datasets often contain missing values, inconsistent formats, and categorical variables that must be converted into numerical form before performing analysis.

Proper data preparation ensures that models and analytical results are **accurate, reliable, and meaningful**.

In this tutorial, you will learn how to:

- Handle missing values in datasets
- Encode categorical variables into numerical format
- Normalize and standardize numerical data
- Prepare datasets for machine learning and statistical analysis

2. Handling Missing Values

Missing data occurs when **no value is stored for a variable in an observation**. Missing values can negatively impact data analysis and machine learning models if not handled properly.

Common strategies for handling missing data include:

- Removing rows or columns containing missing values
- Replacing missing values with **mean, median, or mode**
- Using **forward fill or backward fill methods**

2.1 Example Dataset with Missing Values

Example dataset containing missing values:

Name	Age	Score
Alice	23	85
Bob	NaN	90
Charlie	25	NaN
David	22	88
Emma	NaN	92

Equivalent Pandas code:

```
import pandas as pd
import numpy as np
data = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emma'],
    'Age': [23, np.nan, 25, 22, np.nan],
    'Score': [85, 90, np.nan, 88, 92]
})
print(data)
```

2.2 Removing Missing Values

The `dropna()` function removes rows that contain missing values.

```
clean_data = data.dropna()
print(clean_data)
```

Output:

Name	Age	Score
Alice	23	85
David	22	88

2.3 Filling Missing Values

Missing values can also be replaced with **statistical values such as mean or median**.

```
data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Score'].fillna(data['Score'].median(), inplace=True)
print(data)
```

This method keeps all rows while filling missing values with reasonable estimates.

3. Encoding Categorical Data

Many datasets contain **categorical variables** such as gender, department, or country. However, machine learning algorithms require **numerical input**. Therefore, categorical data must be converted into numbers.

Two common encoding techniques are:

- Label Encoding
- One-Hot Encoding

3.1 Label Encoding

Label encoding converts categories into **numeric labels**.

Example dataset:

Name	Department
Alice	IT
Bob	HR
Charlie	IT
David	Finance

Pandas code:

```
data = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Department': ['IT', 'HR', 'IT', 'Finance']
})
data['Department_Code'] =
data['Department'].astype('category').cat.codes
print(data)
```

Output:

Name	Department	Department_Code
Alice	IT	1
Bob	HR	0
Charlie	IT	1
David	Finance	2

3.2 One-Hot Encoding

One-hot encoding creates **binary columns for each category**.

Example:

```
encoded = pd.get_dummies(data['Department'])
print(encoded)
```

Output:

	HR	IT
0	0	1
1	0	0
2	0	1
3	1	0

This technique is widely used in **machine learning pipelines**.

4. Normalization Techniques

Normalization rescales numerical data so that values fall within a **specific range**, usually between **0 and 1**.

This helps machine learning models perform better when features have different scales.

4.1 Min-Max Normalization

Min-Max normalization transforms values between **0 and 1**.

Example dataset:

Salary
30000
50000
70000
90000

Pandas code:

```
data = pd.DataFrame({  
    'Salary': [30000, 50000, 70000, 90000]  
})  
data['Normalized'] = (data['Salary'] -  
data['Salary'].min()) / (data['Salary'].max() -  
data['Salary'].min())  
print(data)
```

Output:

Salary	Normalized
30000	0.00
50000	0.33
70000	0.67
90000	1.00

4.2 Standardization (Z-score Normalization)

Standardization transforms values so that the dataset has:

- Mean = 0
- Standard deviation = 1

Pandas code:

```
data['Standardized'] = (data['Salary'] -  
data['Salary'].mean()) / data['Salary'].std()  
print(data)
```

This technique is widely used in **machine learning algorithms** such as **logistic regression** and **SVM**.

5. Practical Example: Combined Data Cleaning

Example combining multiple preprocessing steps:

```
import pandas as pd
import numpy as np
data = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Department': ['IT', 'HR', 'IT', 'Finance'],
    'Salary': [50000, 60000, np.nan, 55000]
})
data['Salary'].fillna(data['Salary'].mean(), inplace=True)
encoded = pd.get_dummies(data['Department'])
data = pd.concat([data, encoded], axis=1)
print(data)
```

This example demonstrates:

- Handling missing values
- Encoding categorical variables
- Preparing the dataset for analysis

6. Exercises

Exercise 1

Create a dataset containing missing values and remove them using `dropna()`.

Exercise 2

Replace missing values in a dataset using **mean or median values**.

Exercise 3

Create a dataset containing a categorical column (e.g., Department or Gender) and apply **label encoding**.

Exercise 4

Convert a categorical column using **one-hot encoding**.

Exercise 5

Normalize a numerical column using **Min-Max normalization**.

Exercise 6

Standardize a dataset using **mean and standard deviation**.